Web Development Curriculum

Complete Theory, Workflows, Developer Tools & Exercise Guidance

Curriculum Scope: Projects 00 through 04 (Linux, Git, HTML, CSS, JavaScript, Portfolio)

Target Audience: Beginner to Intermediate Developers (B1 English Level)

Core Methodology: Peer-to-Peer Learning, Zero Shortcuts, Hands-on Execution

Author: Curriculum Engineering & Pedagogical Design Team

Table of Contents

1. Purpose & Pedagogical Architecture	Philosophy, B1 language, progression	2
2. Core Development Workflows	Terminal, Git/GitHub, Web pipeline	3
3. Browser Developer Tools Mastery	Elements, Styles, Console, Network	4
4. Project 00: Linux + Git + GitHub	Shell commands, remotes, SSH	5
5. Project 01: HTML Fundamentals	Structure, semantics, tables, forms	6
6. Project 02: CSS & Visual Layouts	Flexbox, Box Model, Position, Shapes	7
7. Project 03: JavaScript & DOM	Variables, functions, events, state	8
8. Project 04: Personal Portfolio	Capstone design, theme, bounces	9
9. Evaluation & Peer Defense Guide	Assessment criteria & questions	10

1. Purpose & Pedagogical Architecture

Welcome to the 42 Web Curriculum. This guide is your theoretical companion across all five projects of the Discovery Web Piscine. It is designed to explain the 'why' behind every concept, teach essential terminal and coding workflows, and provide practical hints without spoiling the solutions.

The Learning Progression: Beginner to Intermediate

Every project in this curriculum adheres to a strict pedagogical sequence: **Beginner -> Beginner+ -> Intermediate**. You start with zero assumptions about prior coding knowledge and build competence through direct, tactile practice:

- **Project 00 (Environment & Version Control):** You master the Linux terminal, command-line file manipulation, Git version control, and GitHub collaboration. This creates the professional environment where all future code lives.
- **Project 01 (HTML Structure):** You learn semantic markup, document architecture, text hierarchies, media embedding, structured tables, and accessible user input forms.
- **Project 02 (CSS Presentation):** You learn how stylesheets control the visual appearance of HTML: the Box Model, Flexbox layouts, positioning contexts, pure CSS geometry, and color theory.
- **Project 03 (JavaScript Interactivity):** You introduce computational logic: variables, functions, conditions, manipulating the live Document Object Model (DOM), and reacting to user events.
- **Project 04 (Portfolio Capstone):** You synthesize all previous disciplines to build and deploy an authentic, responsive personal portfolio website from scratch.

Clear B1 English Standard

Software development is a global profession. All instructions, exercises, and explanations in this curriculum are written in clear, concise English at the B1 international proficiency level. Jargon is introduced with clear definitions, sentence structures are direct, and conceptual analogies are grounded in practical reality.

The 42 Peer-Learning Philosophy

In this curriculum, there are no teachers giving lectures. You learn through three pillars:

1. **Autonomy:** Reading official documentation (MDN, man pages, Git documentation) and conducting rigorous self-testing.
2. **Peer Collaboration:** Asking peers for alternative viewpoints, discussing challenging bugs, and explaining your code to others.
3. **Rigorous Defense:** During evaluations, you must defend every line of code. Copying code without deep understanding results in immediate failure.

2. Core Development Workflows

Professional web development relies on standardized, repeatable workflows. Mastering these workflows prevents lost work, confusing merge conflicts, and unmaintainable codebases.

The Git & GitHub Daily Lifecycle

Git is a distributed version control system that records changes to a set of files over time. It functions as a timeline with snapshots (commits) that you can inspect, compare, and revert.

```
Edit Source Files (HTML, CSS, JS)
   ■
   ▼
  git status <-- ALWAYS check what changed!
   ■
   ▼
  git add <-- Stage specific verified files
   ■
   ▼
  git commit -m "feat:..." <-- Record snapshot with meaningful description
   ■
   ▼
  git push origin main <-- Publish snapshots to remote GitHub repository
```

Key Git Rules:

- **Never use git add . blindly:** Review files with git status and git diff first to avoid committing secret keys, temporary files, or unwanted changes.
- **Write descriptive commit messages:** Use conventional prefixes such as feat: (new feature), fix: (bug fix), docs: (documentation), and style: (formatting).
- **Push frequently:** Push your verified commits at the end of each exercise to keep your remote backup synchronized.

The Web Development Pipeline: HTML -> CSS -> JavaScript

When building any web feature, always follow this three-step architectural sequence:

1. **HTML First (Content & Hierarchy):** Write the raw HTML structure. Ensure headings, paragraphs, buttons, and inputs make logical sense even with zero CSS styling.
2. **CSS Second (Layout & Aesthetics):** Apply CSS to position elements (Flexbox/Grid), set typography, define colors, and establish responsive media queries.
3. **JavaScript Third (Behavior & Interactivity):** Attach event listeners to user controls, manipulate DOM properties, and manage dynamic state changes.

3. Browser Developer Tools Mastery

Modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) include powerful Developer Tools (DevTools). A developer who cannot use DevTools is coding blindfolded. Learn to use them daily.

How to Open Developer Tools:

- Press F12 on your keyboard, OR press Ctrl + Shift + I (Windows/Linux), OR right-click anywhere on a webpage and select **Inspect**.

The Essential Panels:

- **Elements Panel (DOM & Styles):** Displays the live HTML tree in memory. You can click any element to see its computed dimensions, active CSS rules, and inherited properties. You can double-click HTML to edit tags live or check/uncheck CSS rules to test visual changes without modifying your source files.
- **The Box Model Diagram:** Located at the bottom of the Styles panel. It visually shows the exact pixel measurements of an element's Content, Padding, Border, and Margin. Use this to instantly debug unwanted spacing.
- **Console Panel:** Displays JavaScript log messages (console.log()), runtime warnings, and error stack traces. You can type JavaScript commands directly into the prompt to test functions or query DOM nodes live.
- **Network Panel:** Shows every file requested by your webpage (HTML, CSS files, images, scripts). If an image is broken, check the Network panel for a 404 Not Found status code to fix the file path.
- **Device Toolbar (Responsive Mode):** Click the mobile/tablet icon (Ctrl + Shift + M) to simulate various screen sizes (iPhone, iPad, desktop) and verify responsive media queries.

Standard Debugging Workflow:

1. If an element looks misaligned -> Inspect Element -> Check Box Model & Flexbox properties.
2. If a button does not react -> Open Console -> Check for red JavaScript syntax or reference errors.
3. If an image does not display -> Open Network Tab -> Inspect relative file path and casing.

4. Project 00 Theory & Guidance: Linux + Git + GitHub

Core Theory: The Linux Shell & Filesystem

In Linux, everything is represented as a file or stream. The filesystem is organized as a single hierarchical tree starting at the root directory (/).

- **Absolute Path:** Begins at the root: /home/student/workspace/project00.
- **Relative Path:** Relative to your current working directory: docs/notes.txt or ../backup.

Essential Command Summary:

Command	Description	Example Usage
`pwd`	Print Working Directory	`pwd`
`ls -la`	List all files including hidden with permissions	`ls -la docs/`
`cd <dir>`	Change directory (use .. for parent)	`cd workspace`
`mkdir -p`	Create directory and parent folders	`mkdir -p a/b/c`
`touch <file>`	Create empty file or update timestamp	`touch index.html`
`cp <src> <dst>`	Copy file or folder (use -r for folders)	`cp a.txt b.txt`
`mv <src> <dst>`	Move or rename file	`mv old.txt new.txt`
`rm <file>`	Remove file (permanent deletion)	`rm notes.txt`
`cat <file>`	Concatenate and print file content	`cat README.md`
`echo "text" > file`	Write text to file (> overwrite, >> append)	`echo 'hi' > a.txt`

SSH Key Authentication Theory:

SSH uses asymmetric public-key cryptography. You generate two matching keys:

1. **Private Key (id_ed25519):** Kept secret on your local machine. Never copy or send it.
2. **Public Key (id_ed25519.pub):** Uploaded to remote servers like GitHub. The remote server challenges your machine, and your local private key proves identity mathematically without sending your password over the network.

5. Project 01 Theory & Guidance: HTML Fundamentals

The Semantic HTML5 Philosophy

HTML provides semantic meaning, not visual style. Using tags like <article>, <nav>, and <header> tells search engines, screen readers, and web browsers what the content represents.

Core Element Reference:

- **Document Skeleton:** <!DOCTYPE html> informs the browser to render in standard HTML5 mode. <head> holds meta tags (charset, viewport, title). <body> holds all visible content.
- **Text Hierarchy:** <h1> (document title, only one per page), <h2> (major sections), <h3> (subsections). Use <p> for body paragraphs. Use for importance and for emphasis.
- **Hyperlinks & Media:** <a href="url"> links documents. Use target="_blank" rel="noopener noreferrer" for external tabs. <img src="path" alt="description"> embeds images; always provide descriptive alt text.
- **Lists & Tables:** Use <ul> for bulleted lists, <ol> for numbered steps. Tables must use <table>, <caption>, <thead> with <th scope="col">, and <tbody> with <td>.
- **Forms & Validation:** Use <form action="#" method="POST">. Pair every input with a <label for="inputId">. Use HTML5 validation attributes: required, minlength, type="email".

Common Mistakes in HTML:

- Skipping heading levels just to change font size (use CSS for sizing).
- Putting <title> inside <body> instead of <head>.
- Using hardcoded Windows paths (like C:\photo.jpg) instead of relative paths (assets/photo.jpg).

6. Project 02 Theory & Guidance: CSS & Visual Presentation

The CSS Box Model Architecture

Every HTML element on a webpage is a rectangular box consisting of four concentric layers:

1. **Content:** The inner text, image, or child elements.
2. **Padding:** Transparent space between the content and the border.
3. **Border:** The line surrounding the padding.
4. **Margin:** Space outside the border separating the element from neighbors.

Under default CSS (box-sizing: content-box), adding padding or borders increases the total element width. Always use **box-sizing: border-box** so that declared widths absorb padding and borders predictably.

Modern Flexbox Layouts:

Flexbox arranges items along a 1-dimensional axis:

- **Container:** display: flex activates flex behavior.
- **Main Axis:** Controlled by justify-content (e.g. space-between, center).
- **Cross Axis:** Controlled by align-items (e.g. center, stretch).
- **Flex Direction:** row (horizontal) or column (vertical).
- **Gutters:** Use gap: 20px for clean spacing without manual margins.

CSS Positioning Contexts:

- static: Default document flow.
- relative: Offsets relative to normal position, and creates a positioning context for absolute children.
- absolute: Removed from flow, positioned relative to nearest positioned ancestor.
- fixed: Positioned relative to the browser viewport; stays pinned during scrolling.

7. Project 03 Theory & Guidance: JavaScript & DOM

The Live Document Object Model (DOM)

When a browser loads an HTML document, it builds an in-memory tree of JavaScript objects called the Document Object Model. JavaScript can inspect, add, modify, and delete nodes on this tree in real time.

Essential DOM Operations:

- **Selection:** document.getElementById('id') and document.querySelector('.class').
- **Text Modification:** element.textContent = 'New text'.
- **Class Manipulation:** element.classList.add('active'), classList.remove('active'), classList.toggle('dark').
- **Dynamic Creation:** const li = document.createElement('li'); parent.appendChild(li);.

The Event-Driven Model:

Web applications are event-driven. Code waits for user interactions (clicks, keypresses, input changes) and triggers callback functions:

```
const myButton = document.getElementById('submit-btn');

  // Listen for the 'click' event
  myButton.addEventListener('click', function(event) {
   // 1. Read input state
   // 2. Perform calculation or validation
   // 3. Update DOM text or styles
   console.log('Button clicked successfully!');
  });
```

8. Project 04 Theory & Guidance: Personal Portfolio Capstone

Capstone Architecture

Project 04 is the culmination of your journey. You synthesize Linux, Git, HTML5, CSS3, and JavaScript into a unified, responsive personal portfolio. You must design and build every component yourself without copying pre-made templates or CSS frameworks.

Mandatory Feature: Interactive Background / Theme Switcher

The requirement to change background color via JavaScript tests your grasp of the complete event pipeline:

1. **User Action:** User clicks the theme button or color palette.
2. **Event Registration:** addEventListener('click', toggleTheme) fires.
3. **JavaScript Logic:** The handler function reads current state or color value.
4. **DOM Mutation:** JavaScript updates document.body.style.backgroundColor or toggles a .dark-mode class.

Guidance for Bounces:

- **Bounce 01 (Profile Photo Switcher):** Store image sources in an array or dataset. When clicked, update img.src and img.alt. Apply a CSS transition: opacity 0.3s for smooth cross-fading.
- **Bounce 02 (English / Arabic Switcher):** Store translated content in a structured JavaScript dictionary: const translations = { en: {...}, ar: {...} };. When switching to Arabic, set document.documentElement.dir = 'rtl' and update text nodes.

9. Evaluation & Peer Defense Guide

Every project ends with a peer evaluation. The evaluation is a technical conversation between two peers. It tests comprehension, not merely whether the output looks pretty on screen.

Evaluator Standard Operating Procedures:

1. **Clone Fresh:** The evaluator clones the repository into an empty directory to verify no uncommitted local files are needed.
2. **Live Inspection:** The student opens the project in a browser and opens Developer Tools.
3. **Conceptual Defense:** For every exercise, ask the student to explain the code. For example:

- 'Why did you choose Flexbox here instead of absolute positioning?'
- 'What happens if we remove box-sizing: border-box from your styles?'
- 'Explain the event lifecycle when this button is clicked.'

4. **Zero Tolerance for Blind Copying:** If the student cannot explain how their code works or why a specific property was used, the exercise must be marked 0.

Final Reminder: Failure is not the opposite of learning; it is the most crucial part of it. Debugging errors, reading documentation, and defending your reasoning will make you a confident, autonomous software engineer.